

# **MASTER OF COMPUTER APPLICATIONS**

## **ADVANCED SOFTWARE ENGINEERING**

### **ASSIGNMENT ON**

## **GIT AND GIT COMMANDS**

**Submitted By:**

Adithyan PR

MCA/827/25

## Table of Contents

|                                                         |                                     |
|---------------------------------------------------------|-------------------------------------|
| <b>TABLE OF CONTENTS .....</b>                          | <b>ERROR! BOOKMARK NOT DEFINED.</b> |
| <b>1. INTRODUCTION TO VERSION CONTROL .....</b>         | <b>4</b>                            |
| 1.1 TYPES OF VERSION CONTROL SYSTEMS.....               | 4                                   |
| <b>2. WHAT IS GIT? .....</b>                            | <b>5</b>                            |
| 2.1 KEY FEATURES OF GIT.....                            | 5                                   |
| <b>3. GIT ARCHITECTURE AND COMPONENTS.....</b>          | <b>6</b>                            |
| 3.1 THE THREE TREES OF GIT.....                         | 6                                   |
| 3.2 GIT OBJECTS .....                                   | 6                                   |
| <b>4. INSTALLING AND CONFIGURING GIT .....</b>          | <b>7</b>                            |
| 4.1 INSTALLATION .....                                  | 7                                   |
| 4.2 INITIAL CONFIGURATION .....                         | 7                                   |
| <b>5. BASIC GIT COMMANDS.....</b>                       | <b>8</b>                            |
| <i>git init</i> .....                                   | 8                                   |
| <i>git clone &lt;url&gt;</i> .....                      | 8                                   |
| <i>git status</i> .....                                 | 8                                   |
| <i>git add &lt;file&gt;</i> .....                       | 8                                   |
| <i>git commit -m "message"</i> .....                    | 8                                   |
| <i>git log</i> .....                                    | 8                                   |
| <b>6. INTERMEDIATE GIT COMMANDS.....</b>                | <b>9</b>                            |
| <i>git branch</i> .....                                 | 9                                   |
| <i>git checkout &lt;branch&gt;</i> .....                | 9                                   |
| <i>git merge &lt;branch&gt;</i> .....                   | 9                                   |
| <i>git remote add &lt;name&gt; &lt;url&gt;</i> .....    | 9                                   |
| <i>git push &lt;remote&gt; &lt;branch&gt;</i> .....     | 9                                   |
| <i>git pull &lt;remote&gt; &lt;branch&gt;</i> .....     | 9                                   |
| <i>git fetch</i> .....                                  | 10                                  |
| <b>7. ADVANCED GIT COMMANDS .....</b>                   | <b>11</b>                           |
| <i>git rebase &lt;branch&gt;</i> .....                  | 11                                  |
| <i>git reset &lt;mode&gt; &lt;commit&gt;</i> .....      | 11                                  |
| <i>git stash</i> .....                                  | 11                                  |
| <i>git cherry-pick &lt;commit&gt;</i> .....             | 11                                  |
| <i>git diff</i> .....                                   | 11                                  |
| <i>git tag</i> .....                                    | 11                                  |
| <b>8. GIT BRANCHING AND MERGING.....</b>                | <b>13</b>                           |
| 8.1 UNDERSTANDING BRANCHES .....                        | 13                                  |
| 8.2 MERGING STRATEGIES.....                             | 13                                  |
| 8.3 BRANCH MANAGEMENT BEST PRACTICES .....              | 13                                  |
| <b>9. PRACTICAL EXAMPLE: COMPLETE GIT WORKFLOW.....</b> | <b>14</b>                           |

|                                                      |           |
|------------------------------------------------------|-----------|
| <i>Step 1: Initialize and Setup</i> .....            | 14        |
| <i>Step 2: Create and Commit Files</i> .....         | 14        |
| <i>Step 3: Create and Work with Branches</i> .....   | 14        |
| <i>Step 4: Merge Branches</i> .....                  | 15        |
| <i>Step 5: Connect to Remote and Push</i> .....      | 15        |
| <i>Step 6: Pull Changes from Remote</i> .....        | 15        |
| <b>10. ADVANTAGES AND DISADVANTAGES OF GIT</b> ..... | <b>16</b> |
| 10.1 ADVANTAGES.....                                 | 16        |
| 10.2 DISADVANTAGES.....                              | 16        |
| <b>11. CONCLUSION</b> .....                          | <b>17</b> |
| <b>12. REFERENCES</b> .....                          | <b>17</b> |

## 1. INTRODUCTION TO VERSION CONTROL

Version control, also known as source control or revision control, is a system that records changes to a file or set of files over time so that specific versions can be recalled later. It is an essential tool in modern software development that enables developers to track modifications, collaborate efficiently, and maintain a complete history of their project.

In software engineering, version control systems (VCS) serve multiple critical purposes. They allow multiple developers to work on the same project simultaneously without overwriting each other's changes. They provide a safety net by enabling developers to revert to previous versions if something goes wrong. Additionally, they maintain a detailed audit trail of who made what changes and when, which is invaluable for debugging and accountability.

### 1.1 Types of Version Control Systems

- **Local Version Control Systems:** These systems keep patch sets (differences between files) on the local disk. They are simple but lack collaboration features. Example: RCS (Revision Control System).
- **Centralized Version Control Systems (CVCS):** These systems have a single central repository that stores all versions. Developers check out files from this central place. Examples include SVN (Subversion) and Perforce. The main drawback is the single point of failure.
- **Distributed Version Control Systems (DVCS):** In these systems, clients fully mirror the repository, including its complete history. This provides redundancy and enables offline work. Git, Mercurial, and Bazaar are examples of DVCS.

## 2. WHAT IS GIT?

Git is a distributed version control system that tracks changes in source code during software development. It was created by Linus Torvalds in 2005 for the development of the Linux kernel. Today, Git is the most widely used version control system in the world, with nearly 95% of developers using it as their primary version control tool.

Git is designed for coordinating work among programmers, but it can be used to track changes in any set of files. Its goals include speed, data integrity, and support for distributed, non-linear workflows. Unlike centralized version control systems, Git allows every developer to have a complete copy of the repository, including its full history, on their local machine.

### 2.1 Key Features of Git

- **Distributed Architecture:** Every developer has a full copy of the repository with complete history, enabling offline work and providing backup redundancy.
- **Branching and Merging:** Git makes creating branches lightweight and fast. Developers can create separate branches for features, experiments, or bug fixes, then merge them back seamlessly.
- **Speed and Performance:** Git is optimized for performance. Most operations are performed locally, making them incredibly fast compared to centralized systems.
- **Data Integrity:** Git uses SHA-1 hashing to ensure the integrity of the data. Every file and commit is checksummed, making it impossible to change history without Git knowing.
- **Staging Area:** Git has an intermediate area called the staging area (or index) where commits can be formatted and reviewed before completion.
- **Free and Open Source:** Git is released under the GNU General Public License version 2.0, making it free to use and modify.

### 3. GIT ARCHITECTURE AND COMPONENTS

Git follows a unique architecture that consists of three main areas where files can reside, along with various components that work together to provide version control functionality.

#### 3.1 The Three Trees of Git

- **Working Directory:** This is the directory on your local file system where you modify files. It contains the actual files you are working on. Changes made here are not yet tracked by Git until you add them to the staging area.
- **Staging Area (Index):** This is an intermediate area where changes are prepared before committing. It allows you to selectively choose which changes to include in your next commit. The staging area gives you control over what goes into each commit.
- **Repository (.git directory):** This is where Git stores the metadata and object database for your project. It contains all the committed snapshots of your project. When you clone a repository, this is what gets copied.

#### 3.2 Git Objects

Git stores data as objects in its database. There are four main types of objects:

- **Blob:** Stores the content of files
- **Tree:** Represents directories and contains references to blobs and other trees
- **Commit:** Points to a tree object and contains metadata like author, message, and parent commits
- **Tag:** Points to a specific commit and provides a permanent reference (used for releases)

## 4. INSTALLING AND CONFIGURING GIT

### 4.1 Installation

Git can be installed on various operating systems:

- **Windows:** Download the installer from [git-scm.com](https://git-scm.com) and run the executable. Follow the installation wizard with recommended settings.
- **Linux (Ubuntu/Debian):** Use the command: `sudo apt-get install git`
- **macOS:** Install via Homebrew: `brew install git`, or download from [git-scm.com](https://git-scm.com)

To verify the installation, open a terminal or command prompt and run:

```
git --version
```

### 4.2 Initial Configuration

After installing Git, configure your identity. This information is used in every commit:

```
git config --global user.name "Your Name"  
git config --global user.email "your.email@example.com"
```

You can also configure the default text editor and check your settings:

```
git config --global core.editor "code --wait"  
git config --list
```

## 5. BASIC GIT COMMANDS

Basic Git commands are essential for day-to-day version control operations. These commands help initialize repositories, track changes, and commit work.

### **git init**

Initializes a new Git repository in the current directory. Creates a hidden .git folder.

*Example:*

```
git init
```

### **git clone <url>**

Creates a local copy of a remote repository with all its history.

*Example:*

```
git clone https://github.com/user/repo.git
```

### **git status**

Displays the state of the working directory and staging area. Shows which changes are staged, unstaged, or untracked.

*Example:*

```
git status
```

### **git add <file>**

Adds file changes to the staging area, preparing them for commit.

*Example:*

```
git add filename.txt  
git add . # adds all files
```

### **git commit -m "message"**

Records the staged changes to the repository with a descriptive message.

*Example:*

```
git commit -m "Add new feature"
```

### **git log**

Displays commit history with details like author, date, and commit message.

*Example:*

```
git log  
git log --oneline # compact view
```



## 6. INTERMEDIATE GIT COMMANDS

Intermediate commands help in collaboration, managing branches, and working with remote repositories.

### **git branch**

Lists, creates, or deletes branches.

*Example:*

```
git branch # list branches
git branch feature-login # create branch
git branch -d old-branch # delete
```

### **git checkout <branch>**

Switches to a different branch or restores files.

*Example:*

```
git checkout feature-login
git checkout -b new-feature # create and switch
```

### **git merge <branch>**

Combines changes from one branch into the current branch.

*Example:*

```
git checkout main
git merge feature-login
```

### **git remote add <name> <url>**

Adds a remote repository connection.

*Example:*

```
git remote add origin https://github.com/user/repo.git
```

### **git push <remote> <branch>**

Uploads local commits to a remote repository.

*Example:*

```
git push origin main
git push -u origin feature
```

### **git pull <remote> <branch>**

Fetches and merges changes from a remote repository.

*Example:*

```
git pull origin main
```

**git fetch**

Downloads objects and refs from remote without merging.

*Example:*

```
git fetch origin
```

## 7. ADVANCED GIT COMMANDS

Advanced Git commands provide powerful capabilities for rewriting history, managing complex workflows, and resolving issues.

### **git rebase <branch>**

Reapplies commits from one branch onto another for a cleaner history.

*Example:*

```
git checkout feature
git rebase main
```

### **git reset <mode> <commit>**

Resets current HEAD to a specified state. Modes: --soft, --mixed, --hard.

*Example:*

```
git reset --soft HEAD~1
git reset --hard origin/main
```

### **git stash**

Temporarily saves uncommitted changes and cleans the working directory.

*Example:*

```
git stash
git stash pop # apply saved changes
git stash list # view stashes
```

### **git cherry-pick <commit>**

Applies changes from a specific commit to the current branch.

*Example:*

```
git cherry-pick abc123
```

### **git diff**

Shows differences between various states of the repository.

*Example:*

```
git diff
git diff --staged
git diff branch1 branch2
```

### **git tag**

Creates, lists, or deletes tags (version markers).

*Example:*

```
git tag v1.0.0  
git tag -a v1.0.0 -m "Release version 1.0"
```

## 8. GIT BRANCHING AND MERGING

Branching is one of Git's most powerful features. It allows developers to diverge from the main line of development and work independently without affecting the main codebase.

### 8.1 Understanding Branches

A branch in Git is simply a lightweight movable pointer to a commit. The default branch is called 'main' (formerly 'master'). When you create a new branch, Git creates a new pointer; it doesn't create a new set of files. This makes branching in Git extremely fast and efficient.

### 8.2 Merging Strategies

- **Fast-Forward Merge:** Occurs when the target branch has not diverged from the source branch. Git simply moves the pointer forward. This is the simplest type of merge.
- **Three-Way Merge:** Happens when both branches have diverged. Git creates a new commit that combines changes from both branches, using the common ancestor as the base.
- **Merge Conflicts:** Occur when Git cannot automatically resolve differences between branches. Developers must manually edit files to resolve conflicts, then add and commit the resolved files.

### 8.3 Branch Management Best Practices

- Keep the main branch stable and deployable at all times
- Create feature branches for new developments
- Use descriptive branch names (e.g., feature/user-authentication)
- Delete branches after merging to keep the repository clean
- Regularly sync your branch with the main branch to avoid large conflicts
- Review code before merging through pull requests

## 9. PRACTICAL EXAMPLE: COMPLETE GIT WORKFLOW

Let's walk through a complete example of using Git in a real-world scenario. We'll create a simple project, make changes, work with branches, and push to a remote repository.

### Step 1: Initialize and Setup

```
# Create a new project directory
mkdir my-project
cd my-project

# Initialize Git repository
git init

# Configure user information
git config user.name "John Doe"
git config user.email "john@example.com"
```

### Step 2: Create and Commit Files

```
# Create a README file
echo "# My Project" > README.md

# Create a Python file
echo "print('Hello, Git!')" > main.py

# Check status
git status

# Add files to staging area
git add README.md main.py

# Commit changes
git commit -m "Initial commit: Add README and main.py"
```

### Step 3: Create and Work with Branches

```
# Create a new feature branch
git branch feature-calculator
git checkout feature-calculator
# Or use: git checkout -b feature-calculator

# Make changes
echo "def add(a, b): return a + b" >> main.py

# Stage and commit
git add main.py
git commit -m "Add calculator function"
```

#### Step 4: Merge Branches

```
# Switch back to main branch
git checkout main

# Merge feature branch
git merge feature-calculator

# Delete the feature branch (optional)
git branch -d feature-calculator
```

#### Step 5: Connect to Remote and Push

```
# Add remote repository
git remote add origin https://github.com/username/my-project.git

# Push to remote
git push -u origin main

# Check remote connections
git remote -v
```

#### Step 6: Pull Changes from Remote

```
# Fetch and merge changes from remote
git pull origin main

# Or separately:
git fetch origin
git merge origin/main
```

## **10. ADVANTAGES AND DISADVANTAGES OF GIT**

### **10.1 Advantages**

- **Distributed System:** Every developer has a full copy of the repository, providing redundancy and enabling offline work.
- **Speed:** Most operations are performed locally, making them extremely fast.
- **Branching and Merging:** Lightweight branches make it easy to experiment and work on multiple features simultaneously.
- **Data Integrity:** SHA-1 hashing ensures that data cannot be corrupted without detection.
- **Large Community:** Extensive documentation, tutorials, and community support.
- **Free and Open Source:** No licensing costs and can be modified to suit specific needs.
- **Industry Standard:** Used by most software development teams worldwide.
- **Powerful Tools:** Extensive ecosystem of GUIs, integrations, and hosting platforms like GitHub, GitLab, and Bitbucket.

### **10.2 Disadvantages**

- **Learning Curve:** Git has many commands and concepts that can be overwhelming for beginners.
- **Binary Files:** Not optimized for large binary files; can make repository size grow quickly.
- **Complex History:** Advanced operations like rebasing can make history complex and confusing.
- **Merge Conflicts:** Can be difficult to resolve, especially for novice users.
- **Command-Line Interface:** While GUIs exist, full power requires command-line knowledge.
- **Accidental Data Loss:** Incorrect use of commands like `git reset --hard` can cause data loss.



## 11. CONCLUSION

Git has revolutionized the way developers collaborate and manage code. As a distributed version control system, it provides unparalleled flexibility, speed, and reliability. From small personal projects to large enterprise applications, Git has become the de facto standard for version control in software development.

Understanding Git commands—from basic operations like init, add, and commit, to advanced techniques like rebase and cherry-pick—is essential for modern software engineers. While the learning curve can be steep, the benefits of mastering Git far outweigh the initial challenges. The ability to track changes, collaborate seamlessly, experiment safely with branches, and maintain code integrity makes Git an indispensable tool in advanced software engineering.

As software development continues to evolve, Git remains at the forefront, adapting to new workflows and paradigms. Whether working individually or as part of a large distributed team, Git empowers developers to build better software through effective version control and collaboration. Mastering Git is not just about learning commands; it's about understanding a philosophy of iterative development, experimentation, and continuous improvement.

## 12. REFERENCES

- [1] Chacon, S., & Straub, B. (2014). Pro Git (2nd ed.). Apress. Available at: <https://git-scm.com/book>
- [2] Git Official Documentation. (2025). Git Reference Manual. Retrieved from <https://git-scm.com/docs>
- [3] Atlassian. (2025). Git Tutorial. Atlassian Git Tutorial. Retrieved from <https://www.atlassian.com/git/tutorials>
- [4] GitHub. (2025). GitHub Docs - About Git. Retrieved from <https://docs.github.com>
- [5] Torvalds, L. (2005). Git - Fast Version Control System. Linux Kernel Archives.
- [6] GeeksforGeeks. (2024). What is Git Version Control? Retrieved from <https://www.geeksforgeeks.org>
- [7] DataCamp. (2025). Top 20 Git Commands with Examples. Retrieved from <https://www.datacamp.com>